AI 海難救助システム

ドローンが AI で遭難者を見つけ、自律フォイルボードが駆けつける

Physical AI 応用講座1 最終課題 ／ omnicampus アカウント: ba2vwbus ／ 2026 年 10 月
動画: https://youtu.be/CmVHoU2eICE(YouTube「PAI最終課題_ba2vwbus」、本編は 2 倍速) ／ コード: https://github.com/ba2vwbus13/pai-ai-sea-rescue-board



図1 動画の画面。左はドローンのカメラ映像と AI の検出結果(赤い枠が人)。右上は Gazebo の 3D 画面で、3 台目のボード B3 が 3 人目の遭難者へ向かっている。右下は捜索マップ(緑の線がボードの航跡)と、救助の進み具合。左下は出来事の記録。

1. 概要

海水浴場で、遊泳者が離岸流に流される事故を想定した。ドローン 1 機と、無人で走るフォイルボード 3 台が協力して救助する。ロボットが行うタスクは次の 4 つである。

1. ドローンが遭難者を**捜す**。ドローンは、遭難者が海流で流された先を予測し、その地点から渦巻き状に捜索する。
2. ドローンが遭難者を**見つける**。カメラ映像を AI(YOLO11m)で調べ、人らしいものを見つけたら、降下してもう一度確かめる。
3. ドローンがボードに位置を**知らせる**。オペレータがボードを発進させたあと、ドローンは遭難者の上空にとどまり、遭難者の位置を LoRa 無線で毎秒ボードへ送る。
4. ボードが遭難者のもとへ**駆けつける**。ボードは、遭難者が流れていく先を予測して先回りし、遭難者の手前 2 m で止まって浮き具の代わりになる。

すべて、PX4(ドローンの飛行制御)・Gazebo(物理シミュレータ)・ROS 2(ロボットのプログラム同士をつなぐ仕組み)の上で動かした。

3 / 3 名	0 件	0.5 m 以内	約 85 ms

3 回の実行すべてで 全員を発見しボード到着	人以外を遭難者と 判定した誤り	停止距離 2 m に対する ボードの到着誤差	ドローン → ボードの 座標伝送の平均遅延

2. 目的

離岸流に流された人は、1 分間に約 15 m 沖へ運ばれる(流速 0.25 m/s のとき)。監視員が見失うと、どこを捜せばよいかわからなくなり、見つけても泳いで助けに行く人が危険にさらされる。

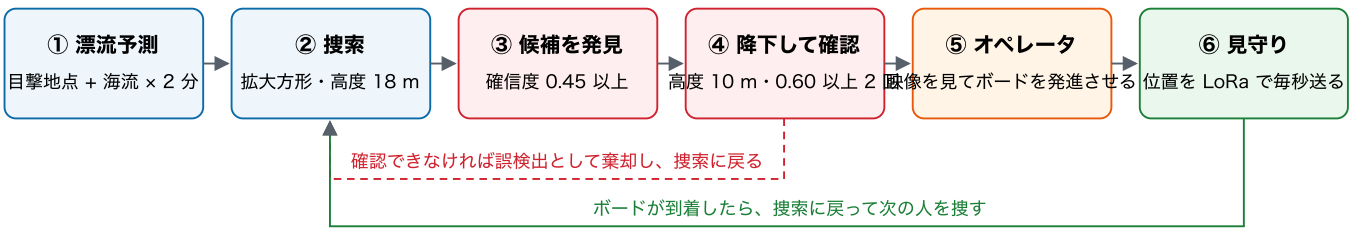
そこで、**人が海に入らずに**、事案発生から **5 分以内**に遭難者のもとへ浮力のあるものを届けることを目標にした。捜索をドローンと AI に任せ、救助をボードに任せることで、監視員は映像を見て判断することに集中できる。

3. システムの構成



図2 システムの構成。Gazebo が世界を、PX4 がドローンの飛行を受け持つ。AI と救助の判断は、自作した ROS 2 のノードが行う。ノード同士はトピックでメッセージをやりとりする。PX4 は ROS 2 のノードではないので、Micro XRCE-DDS(PX4 の中のクライアントと PC 上のエージェントを UDP でつなぐ仕組み)を通して、ROS 2 のトピックと PX4 の中の通信を橋渡しする。

処理の流れ(mission ノードの状態)



全員にボードが到着したら帰還・着陸。ボードは ⑤ で発進し、先回り誘導で手前 2 m に止まる

図3 処理の流れ。ドローンは 1 機なので、確認と見守りの間は捜索が止まる。

4. AI の使い方

物体検出には YOLO11m(COCO で学習済み)を使い、推論だけを行った。ONNX 形式に変換し、onnxruntime で GPU(RTX 3060)上で動かしている。カメラ画像 1280×960 を 3×3 のタイルに分けて推論するので、1 枚の推論に約 230 ms かかる。そこで、推論は毎秒 1 回にした。

AI が出すのは「画像のどこに人がいるか」である。救助に必要なのは「海のどこに人がいるか」なので、次の計算で変換する。

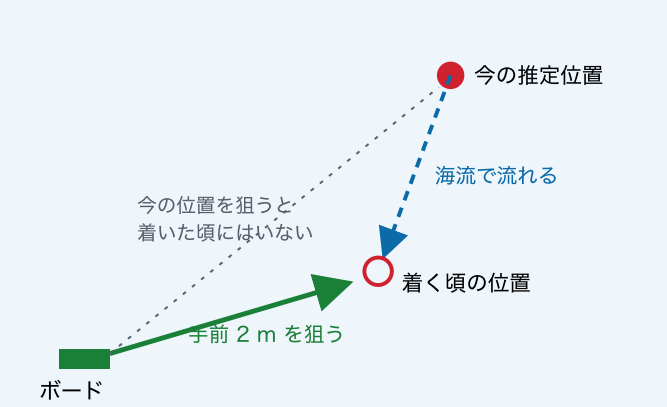
1. 画像の中心からのずれを、カメラの焦点距離 537 画素で割って、カメラから見た方向にする。
2. カメラは前方斜め下 30° を向いているので、その傾きとドローンの姿勢で回転させ、地面に対する方向にする。
3. ドローンの位置からその方向へ線を伸ばし、海面と交わる点を人の位置とする。40 m より遠い点は誤差が大きいので使わない。

5. 工夫した点

表1 工夫した点と、その効果

工夫	内容	効果
漂流予測から捜索を始める	通報は事案発生の 2 分後。その間に遭難者は約 35 m 流されているので、目撃地点ではなく「目撃地点 + 海流 × 経過時間」を中心に、国際基準(IAMSAR)の拡大方形捜索を行う。脚の間隔 32 m は、カメラの見える幅から重なり 25% で決めた。	3 名とも捜索範囲内で発見
タイル分割推論	高度 18 m から見た人は、画像の中でとても小さく写る。画像を 3×3 に分けて拡大してから推論することで、小さな人を見落とさないようにした。	高度 18 m のまま発見できる
2 段階の確認	確信度 0.45 で「候補」とし、高度 10 m まで降りて 0.60 以上が 2 回出たら「確定」とする。確認できなければ棄却して捜索に戻る。	誤検出 0 件
人が最後に判断する	ボードを発進させるかどうかは、AI ではなくオペレータが映像を見て決める。映像は、候補を見つけてからボードを発進させるまでの間だけ地上へ送ると仮定した。映像の伝送そのものは模擬せず、送る時間だけを数えた。	映像を送る時間が飛行時間の 11%
先回り誘導	ボードは、遭難者の今の位置ではなく、「ボードが着くまでに流れていく先」を狙う。ドローンから毎秒届く推定位置・推定時刻・海流から計算する。	到着誤差 0.5 m 以内
時刻を正しく付ける	位置には「写真を受け取った時刻」を付ける。推論が終わった時刻を付けると、その間に流れた分だけ予測がずれるからである。	漂流の予測がずれない
正解は評価だけに使う	遭難者とボードの本当の位置は、評価専用のプログラムだけが記録する。AI と制御は、カメラ・無線で得た情報だけで動く。	公平な評価

先回り誘導のしくみ



ボードが狙う点は次の式で決める。

> 狙う点 = 推定位置 + 海流 ×(今 − 推定時刻 + 到着までの時間)
> そこから、ボードの側へ 2 m 手前の点

到着までの時間は「残りの距離 ÷ 今の速さ」で、毎回計算し直す。方位は P 制御で合わせる。向きのずれに 2.0 を掛けた値を回る速さとし、その上限を 1 rad/s とする。速さが 4 m/s を超えると水中翼で船体が浮き

(離水)、最高 6 m/s で走る。25 m 以内に入ったら減速し、狙う点まで 0.8 m で止まる。止まったあとは浮き具として遭難者と一緒に流れ、発光マーカーを出す。

6. 結果

同じ条件で 3 回実行した。LoRa のパケットロスは実行のたびに違う乱数で決まる。

表2 3 回の実行結果

項目	1 回目	2 回目	3 回目	目標	判定
発見・確定した遭難者(誤り)	3 名(0)	3 名(0)	3 名(0)	全員	達成
見つけた順	P1→P2→P3	P1→P2→P3	P1→P3→P2	—	—
1 人目にボード到着(事案発生から)	175 s	175 s	175 s	300 s 以内	達成
2 人目にボード到着	234 s	266 s	240 s		達成
3 人目にボード到着	417 s	417 s	326 s		未達
ボードの航行時間(1・2・3 人目)	26・32・44 s	26・33・44 s	26・36・36 s	—	—
ボードと遭難者の距離(停止距離 2 m)	1.93〜2.23 m	1.58〜2.16 m	1.69〜2.24 m	誤差 0.5 m 以内	達成
LoRa の受信数 / 送信数	94 / 98	95 / 99	93 / 94	—	—
LoRa の遅延(平均 / 最大)	85.0 / 86.7 ms	86.7 / 89.8 ms	82.5 / 86.4 ms	100 ms 以内	達成
映像を送った時間 / 飛行時間	38 / 343 s	38 / 346 s	38 / 247 s	—	—

映像を送る時間は、飛行時間の 11% だった。常に送る場合と比べて、送る時間は 89% 短い。ただし、映像のビットレート・遅れ・途切れは模擬しておらず、送る時間を数えただけである。

1 回目と 2 回目は P1 → P2 → P3 の順に見つけ、3 人目の到着は 417 s だった。3 回目は、1 人目のボードが着いた直後に P3 がカメラに入り、P2 より先に見つけたので、3 人目の到着は 326 s と 91 s 早かった。どの人がいつカメラに写るかの小さな違いで、見つける順番と全体の時間が変わる。ボードは毎秒届く座標で誘導を続けるので、LoRa のパケットを 1〜4 個失っても、到着の精度は変わらなかった。

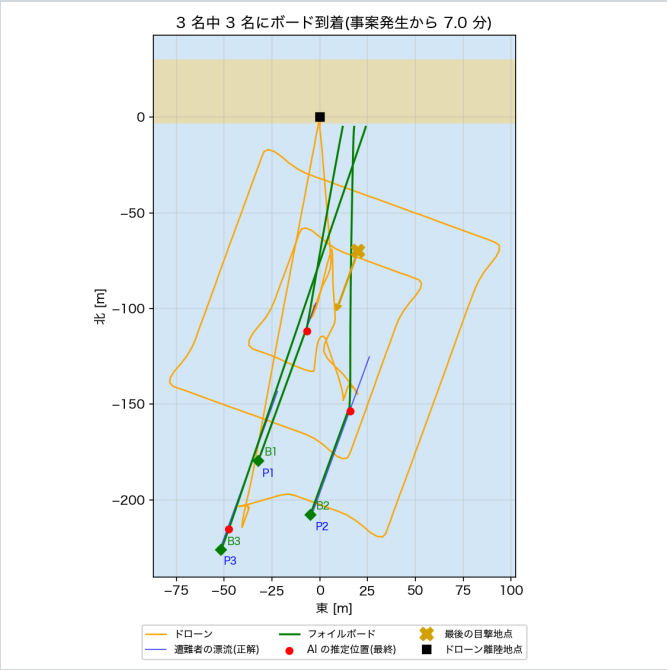


図4 1 回目の軌跡(北が上、上端が砂浜)。オレンジはドローン、緑はボード、青は遭難者の本当の漂流、赤は AI の推定位置。ドローンは漂流予測の中心から渦巻き状に捜索している。

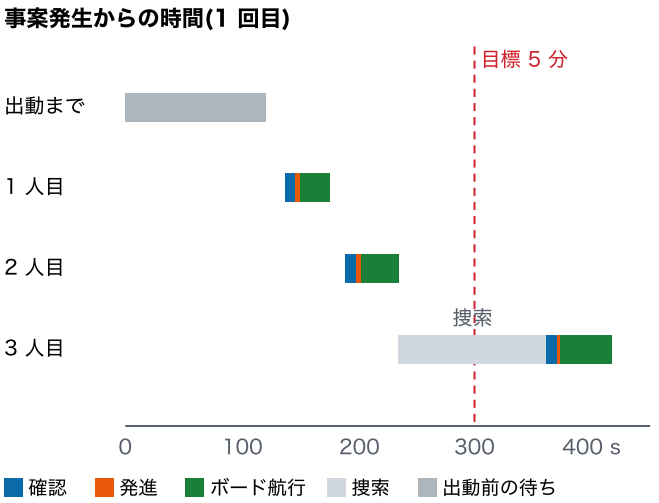


図5 時間の内訳(1 回目)。出動前の待ちが 120 s、1 人目の発見は 137 s、2 人目は 189 s、3 人目は 361 s。ボードの航行は 26〜44 s と短い。

7. 考察と今後

3 人目の到着が 5 分を超えた理由を、時間の使われ方から調べた。ボードの航行は 44 s 以内であり、救助が遅れた原因ではなかった。時間を使っていたのは、次の 3 つである。1 つ目は、事案発生から出動までの待ち時間 120 s である。2 つ目は、捜索の外側にいた 3 人目を捜す時間である。3 つ目は、ドローンが 1 機しかないために、1 人目と 2 人目の確認と見守りをしている合計 83 s の間、捜索が止まることである。

この結果から、次の 3 つの改善が考えられる。

- ドローンを 2 機にして、確認と見守りをする役と、捜索を続ける役を分ける。そうすれば、捜索が止まる 83 s がなくなり、3 人目の発見を少なくとも 83 s 早められる。1 回目の 417 s から 83 s を引くと 334 s となり、目標の 5 分(300 s)に近づく。
- 通報と同時にドローンが自動で離陸するようにして、出動までの待ち時間を縮める。今は、目標の 5 分のうち 120 s が出動前に使われている。
- 見守り中に測った遭難者の位置の変化から、実際の海流を推定する機能を加える。今は海流の予報が正しいとしているが、実際の海では予報に誤差があるからである。

8. 制約

ボードの動きは、速さと旋回の制限、離水の高さだけを持つ簡略化したモデルで表した。波・風・浮力・流体の力は計算していない。遭難者の見た目は、Gazebo の人のモデルで代用している。LoRa は、送信時間を Semtech の式で計算し、パケットロス 3% を乱数で与えた模擬である。これらは実機で確かめる必要がある。

提出物は、次の 3 つである。動画 1 本(2 分 50 秒、本編は 2 倍速、YouTube「PAI最終課題 _ba2vwbus」)・コード一式(GitHub 上記 URL。動かし方は README に記載)・本資料。使用したツールは、PX4 v1.18 SITL、Gazebo Harmonic 8、ROS 2 Humble、YOLO11m(ONNX、onnxruntime-gpu)、Ubuntu 22.04 + RTX 3060 である。